

DEBUG AND DEPLOY AN NGINX CONTAINER FOR LOAD BALANCING, A NODE.JS CONTAINER FOR APPLICATION LOGIC, AND REDIS FOR CACHING WITH DOCKER COMPOSE (OPEN SOURCE REPOSITORY)

We will deploy an **NGINX** server for load balancing, a **NODE.JS** application for application logic, and a **REDIS** caching service for quick data access with **Docker Compose**.

As we already know, **Docker Compose** is a tool that allows you to **start and manage multiple Docker containers** together using a **single configuration file**.

We are going to **clone a repo** from **Awesome Compose** for this **walkthrough**.

AWESOME COMPOSE is an **official GitHub repository by Docker** that contains **ready-made Docker Compose examples for different types of application runtimes and technologies**.

In very simple terms, we can think of Awesome Compose as a recipe book.

Instead of writing your own **docker-compose.yml** file from scratch, you can open the recipe book, choose the **type of application** you want (such as **Django, Flask, Node.js, or WordPress**), and use the **provided examples** in the **repo to deploy your application with Docker Compose**.

This is the GitHub repository link for **Awesome Compose**:

<https://github.com/docker/awesome-compose/tree/master>

You can **explore it** and **deploy containers** using any **programming runtime or technology of your choice**.

We are going to clone the **nginx-nodejs-redis** repository from **Awesome Compose** for this **walkthrough**.

This is the link to the repository:

<https://github.com/docker/awesome-compose/tree/master/nginx-nodejs-redis>

As a **DevOps** or **Cloud Engineer**, you do not need to **understand all the syntax** of a **Node.js** runtime, but **knowing how the application is structured is key to containerizing it**, and to

achieve this, you can sit with a **developer** who has **written the code** and **understand how it was built**.

For example, I understand Python, but I can containerize a Node.js application if guided.

```
.
├── README.md
├── compose.yaml
├── nginx
│   ├── Dockerfile
│   └── nginx.conf
└── web
    ├── Dockerfile
    ├── package.json
    └── server.js

2 directories, 7 files
```

The **nginx-nodejs-redis** directory repository is comprised of **two directories** and **seven files**.

I) The **README.md** file just shows you **how to deploy your container**.

II) The **compose.yaml** file creates containers for **nginx**, **Node.js (web 1 and web 2 servers)**, and the **Redis cache database**.

```
services:
  redis:
    image: 'redislabs/redismod'
    ports:
      - '6379:6379'
  web1:
    restart: on-failure
    build: ./web
    hostname: web1
    ports:
      - '81:5000'
  web2:
    restart: on-failure
    build: ./web
    hostname: web2
```

```
ports:
  - '82:5000'
nginx:
  build: ./nginx
  ports:
    - '80:80'
  depends_on:
    - web1
    - web2
```

BREAKDOWN OF EACH CONTAINER SERVICE:

REDIS

```
redis:
  yaml
  image: 'redislabs/redismod'
  ports:
    - '6379:6379'
```

This runs a **Redis container** using the **redislabs/redismod image**

This exposes Redis on its default port **6379**

Redis is typically used for **caching, sessions, or message queuing**

WEB 1 AND WEB 2

```
restart: on-failure
build: ./web
hostname: web1
ports:
  - '81:5000'
```

The **build: ./web** builds the image from the Dockerfile inside the **./web** folder

restart: on-failure automatically restarts the container if it crashes

hostname gives the container a fixed name other containers can use to reach it

Two identical **web app instances** running on **different ports (81 and 82)** but same **Node.js container port 5000**

This is **load balancing** two copies of the **same app** running simultaneously.

NGINX

```
build: ./nginx
ports:
  - '80:80'
depends_on:
  - web1
  - web2
```

This builds **nginx** from a custom **Dockerfile** in **./nginx** meaning **nginx has a custom configuration**.

Nginx listens on **port 80** which is the main **entry point** for all **incoming traffic**.

depends_on waits for **web1** and **web2** to start before **nginx** can start load balancing between these web servers.

III) The **Dockerfile** inside the **nginx directory** is what creates an **nginx image** from a **base nginx image**. It removes the **default configuration code logic** from **nginx** and replaces it with the configuration code logic from the **nginx.config** file created in the **nginx directory**.

IV) The **nginx.conf** file inside the **nginx directory** is what holds the **code logic** for the **nginx load balancer** to **balance traffic** to **web 1** and **web 2 servers**, respectively. **Nginx** listens on **port 80**. This way, the internet can access the **web servers**.

V) The **Dockerfile** inside the **web directory** creates a **Node.js image** from a **Node.js base image**.

VI) The **package.json** file describes the **Node.js** project and tells **npm** how to run it and which packages it needs.

VII) The **server.js** file inside the **web directory** is what holds the **application logic written in Node.js**. It is a **simple code** that allows **Redis** to **cache any data from traffic** and to show **how many times visitors are accessing a website**. **Redis** listens on port **6379**. The **web application** listens on **port 5000**.

We are going to **create two copies** of this same **Node.js** code, and the **actual code** remains the same. We will install this **code twice**. The **first installation** will run on **port 81**, and we will call it **Web 1**. The second installation will run on **port 82**, and we will call it **Web 2**.

STEPS TO DEPLOY USING DOCKER COMPOSE

STEP 1: LAUNCH YOUR DOCKER DESKTOP APPLICATION AND OPEN WSL ON YOUR TERMINAL.

STEP 2: CONFIRM THAT YOU CAN USE DOCKER COMPOSE.

TYPE THE COMMANDS BELOW.

```
docker --version
```

```
docker compose version          # this shows your Docker Compose version
```

STEP 3: CLONE THE nginx-nodejs-redis REPOSITORY FROM AWESOME COMPOSE. BUT CHECK IF YOU HAVE git INSTALLED LOCALLY.

TYPE THE COMMANDS BELOW.

```
git --version          # you should see your git version if you have git installed.
```

```
                        # if git is not installed. install git with sudo apt install git -y
```

```
git clone https://github.com/docker/awesome-compose.git    # clone the full repository
```

from awesome-compose

ls **# this will show all repositories, directories and files.**

Look for the awesome-compose repository.

cd awesome-compose **# change to the awesome-compose repository**

pwd **# ensure you are in the awesome-compose repository**

ls **# list all sub-repos in the awesome-compose repository**

look for the nginx-nodejs-redis sub-repo and change to that

sub-repo

cd nginx-nodejs-redis **# change to the nginx-nodejs-redis sub-repo**

pwd **# ensure you are in the nginx-nodejs-redis sub-repo**

ls **# list everything you have in the nginx-nodejs-redis sub-repo**

STEP 4: START THE CONTAINERS WITH DOCKER COMPOSE AND OBSERVE YOUR CONTAINERS' INTERACTIONS.

TYPE THE COMMANDS BELOW.

```
docker compose up          # it starts all the containers stated in the compose.yaml file

                             # observe your results in the terminal to see containers that

                             # are started and running.
```

TRY ACCESSING YOUR CONTAINER WITH LOCALHOST. OPEN A BROWSER AND ACCESS YOUR APPLICATION ON PORT 80. TYPE <http://localhost:80>

IF YOU GET A 502 BAD GATEWAY ERROR ON YOUR WEB BROWSER, IT MEANS THAT THE WEB SERVERS CONTAINERS web1 AND web2 ARE NOT FUNCTIONING.

AS AT THE TIME OF THIS DOCUMENTATION, THE DOCKER COMPOSE TEAM ADDED SOME FILES TO THE nginx-nodejs-redis SUB REPO, WHERE THEY ADDED A FILE .npmrc WITH THE ignore-scripts=true COMMAND INSIDE THIS SCRIPT FOR SECURITY CHECKS. THIS WAS DONE WITH GOOD INTENTION, AND THIS FILE WAS ADDED (COMMITTED) ABOUT 4 MONTHS AGO TO THIS REPO.

THIS IS CAUSING A BUG BECAUSE npm start, WHICH IS THE COMMAND TO START THE web1 AND web2 CONTAINERS AND KEEP THEM RUNNING, IS TREATED AS A SCRIPT AND NOT A COMMAND, ALLOWING AS STARTED IN THE .npmrc FILE, CAUSING THE WEB SERVERS TO EXIT IMMEDIATELY AFTER IT STARTS. THE WEB SERVERS ARE CREATED, BUT THEY DO NOT RUN.

IN THE FUTURE, THE DOCKER COMPOSE TEAM WILL DISCOVER THE BUG FROM PULL REQUESTS, BUT WE CAN FIX IT LOCALLY FROM HERE.

STEP 5: DEBUG THE WEB SERVER ISSUE. LEAVE THE FIRST TERMINAL OPEN AND OPEN A SECOND TERMINAL. TYPE THE COMMANDS BELOW.

```
ls                          # this will show all repositories, directories and files.

                             # Look for the awesome-compose repository.
```

```
cd awesome-compose          # change to the awesome-compose repository
```

`pwd` **# ensure you are in the awesome-compose repository**

`ls` **# list all sub-repos in the awesome-compose repository**
look for the nginx-nodejs-redis sub-repo and change to that
sub-repo

`cd nginx-nodejs-redis` **# change to the nginx-nodejs-redis sub-repo**

`docker ps -a` **# check the exit status of all containers**
if you get EXITED (0), for only web server 1 and web server
2, then it means that and redis and nginx are running, then
you need to fix the issue

SWITCH BACK TO THE FIRST TERMINAL AND HOLD `ctrl + c` TO STOP YOUR CONTAINERS.

MAKE SOME CORRECTIONS TO THE Dockerfile INSIDE THE WEB DIRECTORY.

`ls`

`cd web` **# change to the web directory**

`ls` **# list all the files in this directory**

`nano Dockerfile` **# open the Dockerfile in this directory with nano**

INSIDE THE Dockerfile, REPLACE CMD ["npm", "start"] **WITH** CMD ["node", "server.js"]

SAVE YOUR CHANGES AND REBUILD YOUR WEB SERVER CONTAINERS FROM SCRATCH.

```
docker compose build --no-cache web1 web2
```

this rebuilds your containers

(web1 and web2) without using

a previous build cache

```
docker compose up
```

TRY ACCESSING YOUR CONTAINER WITH LOCALHOST. OPEN A BROWSER AND ACCESS YOUR APPLICATION ON PORT 80. TYPE <http://localhost:80>

REFRESH YOUR PAGE TO SEE THE NGINX LOAD BALANCER CONTAINER DISTRIBUTING TRAFFIC BETWEEN DIFFERENT web1 AND web2 CONTAINER SERVERS AND THE REDIS CONTAINER SERVING AS A DATABASE TO SAVE THE NUMBER OF TIMES VISITORS ARE ACCESSING THE APPLICATION.

ALL THESE ARE DIFFERENT CONTAINERS WORKING AND COMMUNICATING WITH EACH OTHER.

FOR EXAMPLE, IF THE REDIS CONTAINER FAILS, OUR APPLICATION CAN STILL FUNCTION.

HOLD ctrl + c TO STOP YOUR CONTAINERS

```
docker compose down # remove all stopped containers
```

`docker compose ps -a` **# confirm they are all removed**

docker compose down --rmi all

remove all images built by the compose team